

GAME&GO

Gaming Lounge Time & Tab Manager

Software Requirements Specification (SRS)

Team Template & Implementation Reference

Version 3.0 | C++17 OOP + PostgreSQL + libpqxx + Dear ImGui

Prepared for: Software Engineering Team

Development target: Arch Linux / VS Code / CMake

Deadline: October 6, 2026

INTERNAL TECHNICAL DOCUMENTATION - TEAM TEMPLATE

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) documents the complete Game&Go Gaming Lounge Time & Tab Manager as a five-person team project. It is intentionally written as an implementation reference: the supplied C++ files declare the contracts and leave TODO work to the assigned member, while this document explains what each declaration must eventually do.

By reading this document, an engineer should understand:

- the complete C++ object model and which file owns each class
- the exact PostgreSQL baseline tables, columns, relationships and required constraints
- how PostgreSQL rows become shared C++ records and then reach Dear ImGui
- where Factory, Strategy and State patterns are used and what each solves
- how the SOLID principles are expressed in the split-file architecture
- which teammate owns every file and function group
- how the project is expected to connect to PostgreSQL through libpqxx without coupling the UI/core to pqxx
- what must be tested before final integration and demonstration

1.2 Project Overview

Game&Go is a desktop management system for a gaming lounge. It tracks branch locations, users, physical gaming stations, customer reservations and active gaming sessions (tabs). The central business goal is to keep station availability, reservations, session timing and billing consistent while five developers work in parallel.

1.3 Scope

- Branch and station management for PC, PS4 and PS5.
- Admin, Staff and Customer users with role-based permissions.
- Reservation creation and lifecycle management: Pending, Confirmed, Canceled.
- Active sessions with start time, optional end time and final cost.
- Time-based billing using Standard and VIP billing strategies.
- Dear ImGui screens for login, dashboard, stations, reservations and active sessions.
- PostgreSQL CRUD, JOIN, constraints, relationships and transaction work.
- A strict three-tier dependency direction and a Git/Jira workflow designed for parallel work.

1.4 Definitions and Acronyms

Term	Definition
Game&Go	Gaming Lounge Time & Tab Manager
OOP	Object-Oriented Programming
STL	Standard Template Library (C++)
CRUD	Create, Read, Update, Delete
PK	Primary Key
FK	Foreign Key
DIP	Dependency Inversion Principle
SRP	Single Responsibility Principle
OCP	Open/Closed Principle
LSP	Liskov Substitution Principle
ISP	Interface Segregation Principle
GUI	Graphical User Interface
pqxx	C++ library used to access PostgreSQL

2. Functional & Technical Requirements

2.1 Required Technology

Technology	Required Use
C++17	Application language and OOP implementation
CMake	Project configuration and dependency/link management
PostgreSQL	Persistent relational storage
libpqxx	C++ PostgreSQL access layer; isolated to PostgresDB implementation
Dear ImGui	Immediate-mode GUI
GLFW	Window/input backend for ImGui
OpenGL	Rendering backend
Arch Linux + VS Code	Intended development environment

2.2 Required OOP Concepts

- Classes and objects: every major responsibility is represented by a class.
- Encapsulation: data members stay private and are accessed through controlled methods.
- Abstraction: User, IDatabase, IBillingStrategy and IReservationState expose contracts.
- Inheritance: concrete role, strategy and state classes derive from abstractions.
- Polymorphism: callers use base pointers/references without depending on concrete implementations.
- Composition: Reservation owns its current state; SessionManager owns injected application services.
- Separation of responsibilities: no class should own GUI, SQL, billing and reservation logic at the same time.

2.3 Required Database Topics

- table creation and relational design
- SERIAL/generated IDs
- primary and foreign keys
- ON DELETE SET NULL / CASCADE
- NOT NULL and CHECK constraints
- INSERT, SELECT, UPDATE and DELETE
- JOIN queries and aggregate/reporting queries
- PostgreSQL to C++ connectivity with libpqxx
- transaction handling for multi-step session changes

2.4 Required Application Features

Feature	Required Behavior
Branch management	Load and select branches; show branch context in GUI.
User roles	Admin, Staff, Customer records with capability-based UI behavior.
Station management	Show type, hourly rate and status; authorized users may change status.
Reservations	Create, list, confirm and cancel reservations with explicit state transitions.
Active sessions	Start a session, keep it active while end time is null, then finish and store final cost.
Billing	Calculate time-based cost through a swappable billing strategy.
Dashboard	Show operational summaries and active session information.
Error handling	Database/core errors must be surfaced to GUI without crashing the render loop.

3. Architecture and File Separation

3.1 Why the Single main.cpp Was Split

The earlier single-file template was useful as a teaching skeleton, but keeping every class, database declaration, GUI declaration and application controller in one file gives that file many unrelated reasons to change. In this revised version, the project is split into focused headers and source files so SRP, DIP and parallel Git work are easier to apply.

The split is structural, not a change to the database design. The PostgreSQL SQL contract remains the same baseline; only ownership comments in the SQL template were cleaned so the SQL file is solely a Member 1 artifact.

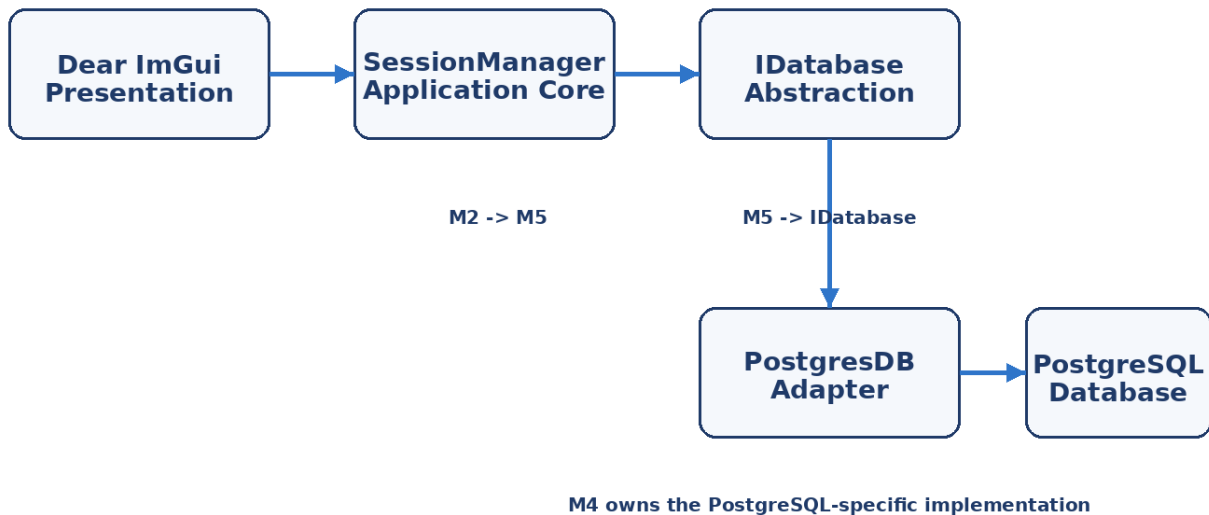


Figure 1 - Required dependency direction. ImGui talks to SessionManager; SessionManager talks to IDatabase; PostgresDB implements IDatabase and is the only class allowed to know PostgreSQL-specific details.

3.2 Three-Tier Responsibility

Tier	Primary Files	Responsibility
Presentation	include/Presentation/ImGuiApp.h src/Presentation/ImGuiApp.cpp	Rendering, widgets, navigation, input and status display.
Application/Core	include/Core/SessionManager.h src/Core/SessionManager.cpp	Use-case orchestration, permission/business validation, pattern coordination.
Domain	include/Domain/* src/Domain/* include/Core/Helpers.h	OOP model, Factory, Strategy, State, time/money rules.
Database	include/Database/* src/Database/PostgresDB.cpp	SQL execution, row mapping, transactions and connection handling.
Persistence	game_and_go_database.sql	PostgreSQL schema, constraints, test data and SQL demonstrations.

3.3 Dependency Rules

- ImGuiApp must depend on SessionManager, not PostgresDB.
- SessionManager must depend on IDatabase, not PostgresDB or pqxx.
- Domain classes must not execute SQL or render ImGui widgets.
- Only PostgresDB.cpp should include <pqxx/pqxx>.
- Shared records stay simple data carriers and contain no GUI/database logic.
- The database schema and C++ record fields must remain synchronized.

3.4 Shared Contract Rule

Shared contracts are the places where the five members meet. Changes to CoreData.h, enum text values, IDatabase signatures or SQL column names must be discussed before merging. This is the main mechanism that prevents integration surprises.

4. File-by-File Reference

File	Owner	What It Contains	Why It Is Separate
include/CoreData.h	Shared	BranchRecord, UserRecord, StationRecord, ReservationRecord, SessionRecord, ActiveSessionView, MoneyCents.	Single neutral data contract; changing it affects multiple layers, so it needs explicit team agreement.
include/Domain/Enums.h	M3	UserRole, StationType, StationStatus, ReservationStatus and conversion declarations.	Domain vocabulary should not be mixed with UI/database code.
include/Core/Helpers.h	M3	Time and money helper declarations.	Reusable core calculations belong outside the GUI/controller.
include/Domain/User.h	M3	User base, AdminUser, StaffUser, CustomerUser, UserFactory.	Role creation and capability behavior are a cohesive OOP responsibility.
include/Domain/Billing.h	M3	BillingInput and billing strategy interfaces/classes.	Pricing is an independent business rule and a Strategy extension point.
include/Domain/Reservation.h	M3	Reservation context and State Pattern classes.	Reservation lifecycle behavior has its own stateful responsibility.
include/Database/IDatabase.h	Shared / M4 implementation	Database operation contract.	High-level code must depend on an abstraction.
include/Database/PostgresDB.h	M4	Concrete DB adapter declaration; no pqxx include.	Keeps PostgreSQL details behind the database boundary.
src/Database/PostgresDB.cpp	M4	Actual pqxx implementation TODOs.	This is the single PostgreSQL-specific C++ implementation area.
include/Core/SessionManager.h	M5	Application controller declaration.	Orchestration has a different reason to change from domain logic or UI.
src/Core/SessionManager.cpp	M5	SessionManager implementation TODOs.	Keeps use-case code away from declarations.
include/Presentation/ImGuiApp.h	M2	GUI class declaration and presentation state.	Presentation changes should not touch domain or DB files.
src/Presentation/ImGuiApp.cpp	M2	GUI implementation TODOs.	Keeps ImGui backend/rendering code isolated.
src/Core/Enums.cpp	M3	Enum helper implementation TODOs.	Small focused compilation unit.
src/Core/Helpers.cpp	M3	Time/money implementation TODOs.	Keeps utility logic reusable and testable.
src/Domain/User.cpp	M3	User hierarchy/Factory implementation TODOs.	Role logic changes should not create merge conflicts in unrelated classes.
src/Domain/Billing.cpp	M3	Billing implementation TODOs.	Billing can change independently from reservation/database logic.
src/Domain/Reservation.cpp	M3	Reservation State implementation TODOs.	State transitions remain isolated.
main.cpp	M5	Composition-root declaration only.	Only final object construction and application startup belong here.
CMakeLists.txt	M5	Build configuration TODOs.	Build/dependency wiring is project integration work.
game_and_go_database.sql	M1	Declaration-only SQL template and database TODOs.	Database work is isolated from C++ implementation work.

5. Enumerations and Shared Data Contract

5.1 Enum Mapping

C++ Enum	Values	Exact PostgreSQL Text
UserRole	Admin, Staff, Customer, Unknown	Admin Staff Customer
StationType	PC, PS4, PS5, Unknown	PC PS4 PS5
StationStatus	Available, InUse, Maintenance, Unknown	Available InUse Maintenance
ReservationStatus	Pending, Confirmed, Canceled, Unknown	Pending Confirmed Canceled

TODO [M3]: Implement conversion and parsing helpers. Unknown database values must never accidentally produce privileged behavior or invalid state transitions.

5.2 Shared Records

Record	Fields	Purpose
BranchRecord	id, branch_name, street_address, district, location_city, location_country	Transfers branch information from database/core to GUI.
UserRecord	id, name, role, phone, optional branch_id	Transfers identity/role data; Factory consumes role.
StationRecord	id, branch_id, type, hourly_rate_cents, status	Represents a physical gaming station.
ReservationRecord	id, user_id, branch_id, station_type, reserved_time, deposit_cents, status	Carries reservation data and status.
SessionRecord	id, station_id, user_id, start_time, optional end_time, optional final_cost_cents	Represents an actual tab/session.
ActiveSessionView	session_id, station_id, station_type, user_id, customer_name, start_time, hourly_rate_cents, elapsed_minutes	JOIN-oriented display record for the active-session screen.

5.3 Money and Time Rules

- PostgreSQL money fields use NUMERIC(10,2).
- C++ uses MoneyCents (std::int64_t) for billing calculations.
- Member 4 converts database numeric text to cents at the DB boundary.
- Member 3 performs billing calculations in cents.
- The baseline SQL uses TIMESTAMP, matching the supplied schema.
- Shared C++ records keep timestamp text; Member 3 owns parsing/elapsed-minute logic and Member 4 owns database mapping.

6. PostgreSQL Database Specification

The SQL file is intentionally not executable. It contains commented declaration shapes and TODOs. Member 1 writes the real DDL and database demonstrations from this section. No C++ file should modify the schema behind Member 1.

6.1 Table: branches

Column	PostgreSQL Type	Rules	C++ Mapping
id	SERIAL	PRIMARY KEY, generated	BranchRecord::id
branch_name	VARCHAR(100)	NOT NULL	branch_name
street_address	VARCHAR(150)	NOT NULL	street_address
district	VARCHAR(100)	NOT NULL	district
location_city	VARCHAR(100)	NOT NULL	location_city
location_country	VARCHAR(100)	NOT NULL	location_country

6.2 Table: users

Column	Type	Rules	C++ Mapping
--------	------	-------	-------------

Column	Type	Rules	C++ Mapping
id	SERIAL	PRIMARY KEY	UserRecord::id
name	VARCHAR(100)	NOT NULL	name
role	VARCHAR(50)	NOT NULL; must be Admin/Staff/Customer	role
phone	VARCHAR(20)	nullable	phone
branch_id	INT	FK -> branches.id; ON DELETE SET NULL	optional branch_id

6.3 Table: stations

Column	Type	Rules	C++ Mapping
id	SERIAL	PRIMARY KEY	StationRecord::id
branch_id	INT	NOT NULL; FK -> branches.id; ON DELETE CASCADE	branch_id
type	VARCHAR(50)	NOT NULL; PC/PS4/PS5	type
hourly_rate	NUMERIC(10,2)	NOT NULL; non-negative	hourly_rate_cents
status	VARCHAR(50)	NOT NULL; Available/InUse/Maintenance	status

6.4 Table: reservations

Column	Type	Rules	C++ Mapping
id	SERIAL	PRIMARY KEY	ReservationRecord::id
user_id	INT	NOT NULL; FK -> users.id; ON DELETE CASCADE	user_id
branch_id	INT	NOT NULL; FK -> branches.id; ON DELETE CASCADE	branch_id
station_type	VARCHAR(50)	NOT NULL; PC/PS4/PS5	station_type
reserved_time	TIMESTAMP	NOT NULL	reserved_time
deposit_amount	NUMERIC(10,2)	NOT NULL; non-negative	deposit_cents
status	VARCHAR(50)	NOT NULL; Pending/Confirmed/Canceled	status

6.5 Table: sessions

Column	Type	Rules	C++ Mapping
id	SERIAL	PRIMARY KEY	SessionRecord::id
station_id	INT	NOT NULL; FK -> stations.id; ON DELETE CASCADE	station_id
user_id	INT	NOT NULL; FK -> users.id; ON DELETE CASCADE	user_id
start_time	TIMESTAMP	NOT NULL	start_time
end_time	TIMESTAMP	nullable; NULL while active	optional end_time
final_cost	NUMERIC(10,2)	nullable; non-negative when present	optional final_cost_cents

6.6 Relationship Map

Parent	Child	Relationship / Delete Rule
branches	users	1-to-many; users.branch_id -> branches.id ON DELETE SET NULL
branches	stations	1-to-many; stations.branch_id -> branches.id ON DELETE CASCADE
users	reservations	1-to-many; reservations.user_id -> users.id

		ON DELETE CASCADE
branches	reservations	1-to-many; reservations.branch_id -> branches.id ON DELETE CASCADE
stations	sessions	1-to-many; sessions.station_id -> stations.id ON DELETE CASCADE
users	sessions	1-to-many; sessions.user_id -> users.id ON DELETE CASCADE

6.7 Database Implementation Order

- 1. Create/select the development database in pgAdmin.
- 2. Create branches first.
- 3. Create users and stations.
- 4. Create reservations.
- 5. Create sessions.
- 6. Add required constraints and approved indexes.
- 7. Insert test/demo data.
- 8. Run CRUD, JOIN and aggregate queries.
- 9. Verify foreign-key/delete behavior.
- 10. Only then connect the C++ adapter.

6.8 Required SQL Demonstrations

Area	Member 1 Must Demonstrate
INSERT	Branch, users in all roles, stations in all types, reservation and session test records.
SELECT	Branches, users, branch stations, reservations, active sessions.
UPDATE	Station status, reservation status, user phone.
DELETE	Controlled deletion showing FK behavior.
JOIN	Active session view, reservation management view, station-by-branch view.
Aggregate	Station counts by status, reservation counts by status, sum of completed session costs.

6.9 Database Validation

- Every table must have a primary key.
- Required foreign keys and ON DELETE rules must match the table specification exactly.
- Invalid role/type/status values must be rejected.
- Required fields must remain NOT NULL.
- Hourly rate, deposit amount and final cost cannot be negative.
- Member 1 must test the final schema using realistic demo records.

7. C++ Class Specifications

All classes below are declarations in the split template. The matching .cpp files contain TODO-only implementation guidance. The function descriptions below are the source of truth for what each function should accomplish.

7.1 User / AdminUser / StaffUser / CustomerUser

Implementation owner: Member 3.

Function	Return	Implementation requirement
User(record)	constructor	Copy the shared UserRecord into private state; no permission logic in base constructor.
getId / getName / getPhone / getBranchId / getRole()	value/reference	Return encapsulated values; do not expose mutable internal storage.
canManageStations()	bool	Each concrete role defines station-

		management capability.
canManageUsers()	bool	Admin-only management according to the final team permission matrix.
canManageReservations()	bool	Define operational reservation capability by role.
canViewReports()	bool	Define reporting capability by role.
canUseCustomerFeatures()	bool	Define customer-facing feature capability by role.
AdminUser / StaffUser / CustomerUser(record)	constructor	Forward record to User and establish concrete role behavior.

7.2 UserFactory

Implementation owner: Member 3.

Function	Return	Implementation requirement
UserFactory::create(record)	unique_ptr<User>	Read record.role; create AdminUser, StaffUser or CustomerUser; reject unsupported roles safely. This is the Factory Pattern entry point.

7.3 Billing Strategy

Implementation owner: Member 3.

Function	Return	Implementation requirement
IBillingStrategy::calculate(input)	MoneyCents	Pure pricing contract. Must not access database or GUI.
IBillingStrategy::name()	string	Return human-readable strategy label for GUI/reporting.
StandardBillingStrategy::calculate(input)	MoneyCents	Compute standard cost from hourly rate and elapsed minutes using the agreed partial-hour rule.
StandardBillingStrategy::name()	string	Return the agreed standard label.
VipBillingStrategy(discount)	constructor	Store the approved VIP discount percentage.
VipBillingStrategy::calculate(input)	MoneyCents	Calculate standard base cost and then apply the VIP discount rule.
VipBillingStrategy::name()	string	Return a clear VIP label.

7.4 Reservation State Pattern

Implementation owner: Member 3.

Function	Return	Implementation requirement
IReservationState::status()	ReservationStatus	Return the exact status represented by the concrete state object.
IReservationState::confirm(reservation)	bool	Accept or reject a confirm action according to the current state.
IReservationState::cancel(reservation)	bool	Accept or reject a cancel action according to the current state.
PendingState::confirm()	bool	Perform Pending -> Confirmed.
PendingState::cancel()	bool	Perform Pending -> Canceled.
PendingState::status()	ReservationStatus	Return Pending.
ConfirmedState::confirm()	bool	Reject second confirmation.
ConfirmedState::cancel()	bool	Perform Confirmed -> Canceled.
ConfirmedState::status()	ReservationStatus	Return Confirmed.
CanceledState::confirm()	bool	Reject confirmation because state is terminal.
CanceledState::cancel()	bool	Reject or implement the team-approved no-

		op behavior and document it.
CanceledState::status()	ReservationStatus	Return Canceled.
Reservation(record)	constructor	Create the correct state object from record.status.
Reservation::confirm / cancel()	bool	Delegate to state_ rather than using a large status switch.
Reservation::getRecord()	const ReservationRecord&	Provide read-only record data.
Reservation::getStatus()	ReservationStatus	Return current state status.
Reservation::transitionTo(new_state)	void	Replace state_ after a valid transition.

7.5 DatabaseException

Implementation owner: Member 4.

Function	Return	Implementation requirement
DatabaseException(message)	constructor	Wrap lower-level DB failures in an application-level exception type.

7.6 IDatabase

Implementation owner: Member 4.

Function	Return	Implementation requirement
getUserById(user_id)	optional<UserRecord>	SELECT one user by PK; empty optional when not found.
listUsers()	vector<UserRecord>	SELECT all users in predictable order.
listUsersByRole(role)	vector<UserRecord>	SELECT users filtered by exact role.
listBranches()	vector<BranchRecord>	SELECT all branches.
listStations(branch_id, type)	vector<StationRecord>	SELECT stations for a branch; apply optional type filter.
getStationById(station_id)	optional<StationRecord>	SELECT one station; empty when missing.
updateStationStatus(id,status)	bool	UPDATE status and report whether a row changed.
createReservation(record)	int	INSERT reservation and return generated SERIAL id.
getReservationById(id)	optional<ReservationRecord>	SELECT one reservation.
listReservations(branch_id,status)	vector<ReservationRecord>	SELECT branch reservations with optional status filter.
updateReservationStatus(id,status)	bool	UPDATE reservation status after core validation.
listActiveSessions()	vector<ActiveSessionView>	JOIN sessions, stations and users; active means end_time IS NULL.
getSessionById(id)	optional<SessionRecord>	SELECT one session.
startSession(record)	int	Insert session and change station status in one transaction.
finishSession(id,record)	bool	Update session and release station in one transaction.

7.7 PostgresDB

Implementation owner: Member 4.

Function	Return	Implementation requirement
PostgresDB(connection_string)	constructor	Open and validate a PostgreSQL connection.
~PostgresDB()	destructor	Release the connection safely.
All IDatabase overrides	various	Implement using parameterized libpqxx queries; map pqxx rows to shared records; handle NULLs; throw DatabaseException or

return empty/false as specified.

7.8 SessionManager

Implementation owner: Member 5.

Function	Return	Implementation requirement
SessionManager(database)	constructor	Store injected IDatabase; do not construct PostgresDB internally.
loginByUserId(user_id)	bool	Load UserRecord, call UserFactory, store current user, reject invalid role data.
getCurrentUser()	const User*	Return current user without transferring ownership.
setSelectedBranch(id) / getSelectedBranch()	void / optional<int>	Store/read UI branch context.
loadBranches()	vector<BranchRecord>	Delegate branch query to IDatabase.
loadStations(branch,type)	vector<StationRecord>	Delegate station query.
loadReservations(branch,status)	vector<ReservationRecord>	Delegate reservation query.
loadActiveSessions()	vector<ActiveSessionView>	Delegate active-session JOIN query.
createReservation(...)	bool	Validate user/branch/station type and insert reservation.
confirmReservation(id)	bool	Load reservation, invoke State Pattern, persist Confirmed only after valid transition.
cancelReservation(id)	bool	Load reservation, invoke State Pattern, persist Canceled only after valid transition.
startSession(user,station)	bool	Validate permissions and station conditions, create SessionRecord, call IDatabase.
finishSession(session_id)	bool	Load active session/rate, select billing Strategy, calculate final cost, persist completion.
setBillingStrategy(strategy)	void	Replace active strategy object.
previewCost(rate,minutes)	MoneyCents	Calculate cost without database writes.

7.9 ImGuiApp

Implementation owner: Member 2.

Function	Return	Implementation requirement
ImGuiApp(session_manager)	constructor	Store SessionManager reference only.
initialize(window)	bool	Initialize ImGui context, style and backend bindings.
render()	void	Begin frame, render active screen, end frame.
renderLogin()	void	Demonstration login/user-selection UI.
renderDashboard()	void	Show branch summary, station counts, active tabs and allowed report values.
renderStations()	void	Show station cards/table and authorized controls.
renderReservations()	void	Create/list/confirm/cancel UI.
renderActiveSessions()	void	Show elapsed time, customer/station, estimated cost and finish control.
renderRoleControls()	void	Use capability functions from User; do not duplicate role logic.
renderStatusMessages()	void	Show transient/persistent feedback.
shutdown()	void	Release ImGui resources; do not destroy GLFW objects owned by M5.

8. Design Patterns

8.1 Factory Pattern

Question	Game&Go Answer
Where?	UserFactory + AdminUser/StaffUser/CustomerUser.
Input	UserRecord::role from PostgreSQL.
Output	A unique_ptr<User> containing the correct concrete type.
Why?	Centralizes object creation and avoids repeating role construction logic in UI/core.
OOP link	Inheritance, abstraction, polymorphism.
SOLID link	Supports OCP and SRP by keeping construction separate from use.

8.2 Strategy Pattern

Question	Game&Go Answer
Where?	IBillingStrategy, StandardBillingStrategy, VipBillingStrategy.
Input	BillingInput: hourly rate + elapsed minutes.
Output	MoneyCents final cost.
Why?	Pricing rules can be swapped without rewriting SessionManager.
Extension	A future Night/Weekend/Student strategy can be added as a new class.
SOLID link	Strong OCP and DIP use case.

8.3 State Pattern

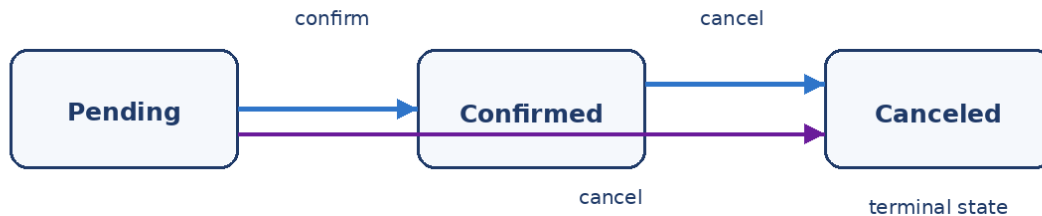


Figure 2 - Reservation lifecycle required by the State Pattern.

Current State	Action	Expected Result
Pending	confirm	Confirmed
Pending	cancel	Canceled
Confirmed	confirm	Rejected
Confirmed	cancel	Canceled
Canceled	confirm	Rejected
Canceled	cancel	Rejected or documented no-op

The SRS requires `status()` in every concrete state because the state object must be able to report which ReservationStatus it represents. Member 3 should implement `PendingState::status()` -> Pending, `ConfirmedState::status()` -> Confirmed, and `CanceledState::status()` -> Canceled.

9. SOLID Principles

Principle	Game&Go Application	What the Team Should Explain
SRP	ImGuiApp renders; SessionManager orchestrates; PostgresDB accesses DB; billing strategies calculate cost; reservation states manage transitions; shared records carry data.	Each class/file has one primary reason to change.
OCP	User subclasses, billing strategies and reservation states are extension points.	New behavior should be added behind abstractions where appropriate.
LSP	AdminUser/StaffUser/CustomerUser are usable through User; concrete strategies through IBillingStrategy; states through IReservationState.	Derived classes must honor base contracts.
ISP	IDatabase exposes focused application operations instead of raw arbitrary SQL.	Consumers depend on the operations they need, not pqxx internals.
DIP	SessionManager depends on IDatabase; ImGuiApp depends on SessionManager.	High-level policy does not depend directly on PostgreSQL.

9.1 Why Splitting Files Helps SRP

Splitting the former single main.cpp is not about creating files for the sake of files. Each new unit has a focused reason to change: billing changes in Billing.cpp, reservation rules in Reservation.cpp, PostgreSQL changes in PostgresDB.cpp, GUI changes in ImGuiApp.cpp, and application use cases in SessionManager.cpp. This lowers merge conflicts and makes each class easier to explain and test.

10. C++ / PostgreSQL / ImGui Integration

10.1 One Complete Action: Finish Session

- 1. M2 ImGui detects the Finish button click.
- 2. ImGuiApp calls `SessionManager::finishSession(session_id)`.
- 3. M5 loads the active session through IDatabase.
- 4. M5 obtains the station rate and elapsed time.
- 5. M5 selects the current billing strategy from M3 and calculates MoneyCents.
- 6. M5 sends the completed SessionRecord to `IDatabase::finishSession()`.
- 7. M4 PostgresDB performs the SQL transaction through libpqxx.
- 8. PostgreSQL stores end_time/final_cost and the station becomes Available.
- 9. Control returns through SessionManager to ImGui, which shows success/error feedback.

10.2 pqxx Boundary

```
#include <pqxx/pqxx>
// Allowed only in src/Database/PostgresDB.cpp
```

The split architecture improves the earlier single-file situation: the pqxx header is no longer needed in every file. PostgresDB.cpp includes pqxx, while PostgresDB.h contains only the abstract-facing declaration plus a forward declaration of `pqxx::connection`.

10.3 Row Mapping Contract

PostgreSQL Value	C++ Target	Notes
SERIAL / INT	int	IDs are mapped directly.
VARCHAR	std::string	Text and controlled enum strings.
NUMERIC(10,2)	MoneyCents	Convert at database boundary.
nullable INT	std::optional<int>	Used by users.branch_id.

nullable TIMESTAMP	std::optional<std::string>	Used by sessions.end_time.
TIMESTAMP	std::string	Used by reservation/session timestamps.

10.4 Transaction Requirements

Operation	Required Atomic Steps
Start session	Verify station -> ensure Available -> insert active session -> set station InUse -> commit; rollback if any step fails.
Finish session	Verify active session -> update end_time/final_cost -> set station Available -> commit; rollback on failure.
Reservation status	Core validates state first -> DB persists new status -> report success/failure.

11. Team Ownership and Git Workflow

Member	Files / Areas	Primary Work
Member 1	game_and_go_database.sql	PostgreSQL database, DDL, constraints, relationships, seed data, CRUD/JOIN/aggregate demonstrations, pgAdmin testing.
Member 2	include/Presentation/ImGuiApp.h src/Presentation/ImGuiApp.cpp	Dear ImGui initialization, screens, widgets, inputs, navigation, role-based rendering, feedback.
Member 3	include/Domain/* src/Domain/* include/Core/Helpers.h src/Core/Helpers.cpp	OOP hierarchy, Factory, Strategy, State, enum parsing, time/money helpers and core business rules.
Member 4	include/Database/* src/Database/PostgresDB.cpp	IDatabase implementation, PostgreSQL connection, libpqxx queries, row mapping, exceptions, transactions.
Member 5	include/Core/SessionManager.h src/Core/SessionManager.cpp main.cpp CMakeLists.txt	Application orchestration, composition root, dependency wiring, CMake, Git integration, end-to-end testing.

11.1 Shared Files

Shared Contract	Rule
include/CoreData.h	Any field/type change requires team agreement because DB mapping, core logic and GUI can all depend on it.
Domain enum text values	Keep exact synchronization with SQL CHECK values and GUI labels.
IDatabase.h	Changing a method signature is an integration-level change; all callers/implementations must be updated together.
SRS_GameAndGo.pdf	Project reference; update it when an accepted contract changes.

11.2 Git Rules

- Work mainly inside the files assigned to you.
- Do not globally reformat unrelated files.
- Keep TODOs until implementation and review are complete.
- Open a focused branch/PR for each logical task.
- Discuss shared-contract changes before merging.
- M5 integrates only after the contributing member has tested their own area.

12. Jira Task Breakdown

12.1 Member 1 - PostgreSQL

- Create development database and confirm connection details.
- Implement five baseline CREATE TABLE statements.
- Implement PK/FK/NOT NULL/CHECK/ON DELETE rules.
- Insert realistic seed/demo records.
- Implement and test CRUD queries.
- Implement and test required JOIN queries.
- Implement aggregate/report queries.
- Test invalid data and foreign-key behavior in pgAdmin.
- Freeze column names/types for M4 integration.

12.2 Member 2 - Dear ImGui

- Initialize ImGui/GLFW/OpenGL.
- Implement login/user-selection screen.
- Implement dashboard.
- Implement stations screen.
- Implement reservation screen.
- Implement active sessions/tabs screen.
- Implement role-based controls using User capability methods.
- Implement status/error messaging.
- Polish navigation/layout for final demo.

12.3 Member 3 - Core/OOP

- Implement enum conversions/parsers.
- Implement time and money helpers.
- Implement User hierarchy and permission behavior.
- Implement UserFactory.
- Implement StandardBillingStrategy.
- Implement VipBillingStrategy.
- Implement Reservation State Pattern.
- Write focused tests for pattern transitions and billing rules.

12.4 Member 4 - Database Integration

- Implement DatabaseException.
- Implement PostgresDB connection lifecycle.
- Implement row-mapping helpers.
- Implement user/branch/station queries.
- Implement reservation queries.
- Implement active/completed session queries.
- Implement start/finish session transactions.
- Test empty results, NULLs and database exceptions.
- Verify every query maps exactly to the shared records.

12.5 Member 5 - Integration/Leadership

- Implement SessionManager constructor and dependency injection.
- Implement login and current-user handling.
- Implement branch/data loading methods.
- Implement reservation use cases with State Pattern.
- Implement session start/finish orchestration with billing Strategy.
- Implement main composition root.
- Complete CMake dependency wiring.
- Merge/test teammates branches.

- Run end-to-end workflow from GUI to PostgreSQL.

13. Build, Run and Environment Requirements

13.1 Development Environment

- Arch Linux
- VS Code
- C++17 compiler
- CMake
- PostgreSQL server/client
- libpqxx development package
- Dear ImGui source or package integration
- GLFW and OpenGL development libraries

13.2 Build Responsibilities

Member 5 owns CMake configuration. The exact imported targets/package names depend on the team's Arch installation and how Dear ImGui sources are added. The CMake template intentionally leaves those dependency-target details as TODOs rather than pretending a package layout is universal.

13.3 Runtime Configuration

The final application must obtain a PostgreSQL connection string through the agreed local development configuration. Do not hard-code private passwords into Git. The concrete connection setup belongs to Member 4, while object construction/wiring belongs to Member 5.

14. Testing and Edge Cases

Area	Minimum Tests
Factory	Admin, Staff, Customer, unknown role.
Billing	Zero minutes, exact hour, partial hour, VIP discount, invalid negative input.
Reservation State	Every valid transition and every invalid transition.
Database	Missing IDs, empty lists, NULL branch_id, invalid values rejected by DB.
Stations	Available -> InUse -> Available; attempt to start when not Available.
Sessions	Active session has NULL end_time/final_cost; completion stores both; transaction rollback is tested.
GUI	Role controls hidden/shown correctly; DB error does not crash render loop.
Integration	Login -> select branch -> view station -> reserve -> confirm -> start -> observe timer -> finish -> see final cost.

14.1 Definition of Done

- Every assigned TODO is implemented and reviewed.
- C++/SQL names and types remain synchronized.
- All required SQL demonstrations work in pgAdmin.
- Application starts and reaches the ImGui dashboard.
- PostgresDB can connect and execute the required operations.
- A complete session lifecycle works end-to-end.
- Reservation state transitions reject invalid actions.
- Billing strategy selection produces the expected cost.
- No GUI/core file directly uses pqxx.

15. Design Notes & Best Practices

Practice	Reason	Game&Go Application
Keep pqxx in PostgresDB.cpp	Prevents PostgreSQL-specific details from spreading.	M4 is the only C++ member who writes pqxx code.
Use interfaces at boundaries	Allows independent testing and replacement.	SessionManager -> IDatabase; User/Strategy/State polymorphism.
Keep records simple	Reduces coupling.	CoreData.h contains data only.
Use cents for money	Avoids binary floating-point rounding issues in billing.	Billing strategies calculate MoneyCents.
Validate in core, enforce in DB	Protects both application behavior and persistent data.	M5 performs use-case validation; M1 adds DB constraints.
Do not duplicate role logic	Avoids inconsistent permissions.	M3 owns capability methods; M2 consumes them.
Use transactions for multi-write operations	Keeps station/session states synchronized.	M4 handles start/finish atomic DB work.

15.1 Common Integration Mistakes to Avoid

- M2 writing SQL directly in ImGuiApp.
- M5 constructing PostgresDB inside a method instead of dependency-injecting IDatabase.
- M4 changing SQL column names to fit a local C++ preference without team agreement.
- M3 putting billing logic into SessionManager instead of the Strategy class.
- M3 using if/else chains in Reservation instead of delegating to state objects.
- Any member renaming shared record fields without updating the mapping contract.

16. Quick Reference Card

Class/File	Primary Responsibility	Pattern / Principle
CoreData.h	Shared data records	Shared contract
Enums.h / Enums.cpp	Controlled vocabulary and parsing	Encapsulation / SRP
Helpers.h / Helpers.cpp	Time and money utilities	SRP
User.h / User.cpp	Roles and permissions	Factory / inheritance / polymorphism
Billing.h / Billing.cpp	Pricing algorithms	Strategy / OCP
Reservation.h / Reservation.cpp	Reservation lifecycle	State / composition
IDatabase.h	Database contract	DIP / ISP
PostgresDB.h/.cpp	PostgreSQL access	DIP boundary
SessionManager.h/.cpp	Use-case orchestration	SRP / DIP
ImGuiApp.h/.cpp	GUI presentation	SRP
main.cpp	Composition root	Dependency wiring
game_and_go_database.sql	Persistent schema and SQL work	Relational design

16.1 Final Dependency Snapshot

ImGuiApp -> SessionManager -> IDatabase <- PostgresDB -> PostgreSQL

Factory: UserRecord -> UserFactory -> concrete User

Strategy: BillingInput -> IBillingStrategy -> concrete billing rule

State: Reservation -> IReservationState -> Pending/Confirmed/Canceled

16.2 Final Ownership Snapshot

Member	Think of the Role As
M1	Build the database.
M2	Build what the user sees.
M3	Build the OOP/business-rule brain.
M4	Build the C++ bridge to PostgreSQL.

17. Submission & Final Meeting Checklist

- The PostgreSQL database is created and populated with development data.
- C++ project builds successfully on the team's Arch machine.
- libpqxx connection works using the agreed configuration.
- Dear ImGui launches successfully.
- At least one end-to-end reservation/session flow is demonstrated.
- Role-based controls are visibly different for Admin/Staff/Customer.
- Billing Strategy and Reservation State Pattern can be demonstrated live.
- The team can point to the exact file owned by each member.
- The SRS matches the final accepted schema and C++ contracts.
- Git/Jira history shows individual work and integration activity.

Appendix A - Declaration Template Rules

The split source package is intentionally unfinished. Header files contain the declarations and ownership/TODO comments. The .cpp files contain TODO implementation guidance only. This prevents the template from turning into a completed solution while still giving every teammate the exact function signature and responsibility.

The PostgreSQL file follows the same philosophy. It is a declaration-only SQL template with commented CREATE TABLE shapes and TODO checklists. The schema contract itself is specified in Section 6 of this SRS.

Appendix B - What Changed in This Revision

- Replaced the previous single-file C++ organization with focused headers/source files to better support SRP and Git parallelism.
- Moved the pqxx include into PostgresDB.cpp instead of exposing it through the whole application.
- Kept the shared CoreData contract as the explicit cross-layer boundary.
- Kept the database design baseline unchanged. Only SQL ownership comments were cleaned so the SQL artifact is M1-only; no database column/table requirement was changed by the split.
- Expanded the SRS file-by-file and function-by-function guidance so a teammate can implement the TODO from the SRS without inventing a different interface.
- Made every State Pattern status() declaration explicitly documented, including the three concrete return values.
- Updated all project naming to Game&Go.